

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN MƯỜI HAI
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã hoàn thiện quy trình kiểm duyệt vi phạm, chuẩn hóa dữ liệu báo cáo và xây dựng nền tảng AI Feedback Dataset trong tuần thứ mười một, tuần thứ mười hai của dự án tập trung vào việc nâng cấp hệ thống theo hướng gần với môi trường production hơn. Trọng tâm chính của tuần này là bổ sung các chức năng quản trị vận hành như Audit Log, Camera Management, Demo Mode, đồng thời tiếp tục cải thiện độ ổn định của pipeline video và AI detection.

Ở các tuần trước, hệ thống đã có khả năng nhận diện, lưu trữ, kiểm duyệt và phân tích dữ liệu vi phạm. Tuy nhiên, để hệ thống có thể vận hành thực tế, cần bổ sung thêm các lớp quản trị quan trọng như quản lý camera, ghi lại lịch sử thao tác người dùng và hỗ trợ chế độ demo ổn định khi không có camera thật.

Các mục tiêu chính trong tuần này bao gồm:

- Xây dựng hệ thống Audit Log để ghi lại các hành động quan trọng trong hệ thống
- Xây dựng module Camera Management để quản lý nguồn camera từ giao diện quản trị
- Chuyển cấu hình camera từ dạng cố định trong file môi trường sang dữ liệu quản lý trong database
- Bổ sung Demo Mode thông qua video mẫu, phục vụ quá trình báo cáo đồ án
- Cập nhật Live Monitoring để sử dụng camera source dạng object có metadata
- Bổ sung snapshot metadata camera vào dữ liệu violation
- Cải thiện pipeline AI nhằm giảm log trùng và hạn chế false positive
- Tối ưu độ ổn định của luồng video khi switch/start/stop camera

Việc hoàn thành các mục tiêu này giúp hệ thống chuyển từ một ứng dụng giám sát AI hoàn chỉnh về mặt chức năng sang một hệ thống có khả năng quản trị, kiểm soát và trình diễn tốt hơn trong môi trường thực tế.

2. Xây dựng hệ thống Audit Log

2.1. Mục tiêu của Audit Log

Trong một hệ thống có nhiều người dùng, nhiều vai trò và nhiều thao tác ảnh hưởng trực tiếp đến dữ liệu, việc chỉ lưu kết quả cuối cùng là chưa đủ. Hệ thống cần có khả năng ghi lại ai đã thực hiện hành động gì, vào thời điểm nào và tác động đến đối tượng nào.

Vì vậy, trong tuần này em đã xây dựng chức năng Audit Log nhằm ghi nhận các hành động quan trọng trong hệ thống.

Các nhóm hành động được ghi log bao gồm:

- User/Admin action
- Violation action
- Alert action
- Camera action

Một số ví dụ về action:

- user.created
- violation.confirmed
- violation.rejected
- violation.deleted
- alert.manual_sent
- camera.created
- camera.updated
- camera.disabled
- camera.force_stopped

Việc chuẩn hóa action theo dạng domain.action giúp dữ liệu log rõ ràng, dễ lọc và dễ mở rộng trong tương lai.

2.2. Thiết kế bảng Audit Log

Audit Log được lưu trong SQL Database với bảng audit_logs.

Các trường quan trọng bao gồm:

- actor_id
- actor_username
- actor_role
- action

- target_type
- target_id
- description
- metadata_json
- ip_address
- created_at

Trong đó, các thông tin về actor được thiết kế nullable. Lý do là trong thực tế không phải log nào cũng đến từ người dùng. Một số hành động có thể đến từ hệ thống, ví dụ như hệ thống tự dừng camera hoặc ghi nhận trạng thái lỗi.

Trường metadata_json là một điểm quan trọng. Thay vì chỉ lưu mô tả dạng text, hệ thống có thể lưu thêm dữ liệu có cấu trúc, ví dụ:

- old_status
- new_status
- camera_id
- rejection_reason
- source_type
- is_demo

Điều này giúp Audit Log không chỉ dùng để đọc thủ công, mà còn có thể phục vụ debug hoặc truy vấn nâng cao sau này.

2.3. API truy xuất Audit Log

Em đã xây dựng API dành riêng cho Admin để truy xuất lịch sử thao tác:

GET /audit-logs

API này hỗ trợ:

- Filter theo actor
- Filter theo action
- Filter theo target type
- Filter theo target id
- Filter theo khoảng thời gian
- Pagination

Việc chỉ cho Admin truy cập Audit Log là phù hợp với yêu cầu bảo mật, vì log có thể chứa các thông tin nhạy cảm liên quan đến thao tác người dùng, camera và dữ liệu hệ thống.

2.4. Giao diện Audit Logs trên Frontend

Ở Frontend, em đã bổ sung trang Audit Logs dành cho Admin.

Trang này bao gồm:

- Bảng danh sách log
- Bộ lọc dữ liệu
- Phân trang
- Modal xem chi tiết metadata

Giao diện được thiết kế đồng bộ với các trang quản trị khác như Violation History, giúp trải nghiệm người dùng nhất quán hơn. Khi cần kiểm tra một thao tác cụ thể, Admin có thể mở modal để xem chi tiết metadata thay vì chỉ xem mô tả ngắn trong bảng.

Chức năng này giúp hệ thống tăng tính minh bạch và hỗ trợ truy vết lỗi tốt hơn trong quá trình vận hành.

3. Xây dựng module Camera Management

3.1. Chuyển camera source sang quản lý trong Database

Trước đây, danh sách camera chủ yếu được cấu hình qua file .env. Cách làm này phù hợp trong giai đoạn phát triển ban đầu, nhưng không thuận tiện khi hệ thống cần quản lý nhiều camera hoặc thay đổi nguồn camera thường xuyên.

Trong tuần này, em đã bổ sung model quản lý camera trong SQL Database.

Thông tin mỗi camera bao gồm:

- id
- code
- name
- location
- source_type
- source_url
- is_active
- last_status

- last_checked_at
- is_deleted
- deleted_at

Trong đó, source_type hỗ trợ ba loại chính:

- webcam
- rtsp
- video_file

Việc đưa camera vào database giúp hệ thống dễ quản lý hơn, đồng thời tạo tiền đề cho việc thao tác camera trực tiếp từ giao diện Admin.

3.2. Hỗ trợ nhiều loại camera source

Hệ thống hiện có thể xử lý nhiều dạng nguồn camera khác nhau.

- Với webcam, source_url sẽ được chuyển đổi sang số nguyên để OpenCV có thể mở thiết bị, ví dụ:
- source_url = 0
- Với RTSP camera, source_url có thể là một đường dẫn stream.
- Với video file, hệ thống sử dụng file video mẫu trong thư mục demo để phục vụ Demo Mode.

Cách thiết kế này giúp Demo Mode trở thành một loại camera source bình thường, thay vì phải xây dựng một pipeline riêng. Đây là một thiết kế sạch hơn vì các luồng xử lý sau đó có thể dùng chung logic camera, switch camera, streaming và violation logging.

3.3. API quản lý camera

Em đã xây dựng các API phục vụ Camera Management, bao gồm:

- Tạo camera
- Cập nhật camera
- Bật/tắt camera
- Xóa mềm camera
- Test connection
- Lấy danh sách camera
- Lấy danh sách demo video

Đặc biệt, API test connection giúp Admin kiểm tra xem nguồn camera có mở được hay không trước khi đưa vào vận hành.

Khi một camera đang được stream mà bị disable hoặc delete, hệ thống sẽ force stop pipeline để tránh tình trạng camera đã bị vô hiệu hóa nhưng stream vẫn tiếp tục chạy.

3.4. Giao diện Camera Management

Ở Frontend, em đã bổ sung trang Camera Management dành cho Admin.

Các chức năng chính bao gồm:

- Xem danh sách camera
- Tạo camera mới
- Chỉnh sửa thông tin camera
- Bật/tắt trạng thái active
- Test connection
- Xóa mềm camera
- Chọn video demo từ danh sách có sẵn

Giao diện này giúp người quản trị có thể quản lý nguồn camera trực tiếp trên web thay vì phải chỉnh sửa thủ công trong file .env.

Đây là một bước nâng cấp quan trọng vì trong môi trường thực tế, hệ thống có thể phải quản lý nhiều camera tại các vị trí khác nhau như cổng chính, bãi xe hoặc khu vực giám sát riêng.

4. Xây dựng Demo Mode

4.1. Demo video như một camera source

Trong tuần này, em đã bổ sung Demo Mode để phục vụ quá trình báo cáo đồ án.

Thay vì xây dựng Demo Mode thành một chức năng riêng biệt, hệ thống xem demo video như một camera source có `source_type=video_file`.

Điều này giúp demo video có thể sử dụng lại toàn bộ pipeline hiện tại:

- OpenCV capture
- YOLO inference
- Tracking
- Violation logging
- WebSocket broadcast

- Traffic stats
- Frontend Live Monitoring

Nhờ đó, khi chạy demo video, hệ thống vẫn hoạt động giống như khi dùng camera thật.

4.2. Giới hạn đường dẫn demo để tránh path traversal

Để đảm bảo an toàn, hệ thống chỉ cho phép video demo nằm trong thư mục:

SourceCode/BE/static/demo

Khi tạo camera dạng video_file, Backend sẽ kiểm tra và chuẩn hóa đường dẫn. Nếu video không nằm trong thư mục demo hợp lệ, hệ thống sẽ từ chối.

Điều này giúp tránh lỗi path traversal, tức là người dùng cố tình truyền đường dẫn ra ngoài thư mục cho phép để đọc file không mong muốn trên server.

5. Cải thiện Violation và AI Pipeline

5.1. Lưu snapshot metadata camera trong violation

Mỗi violation hiện tại không chỉ lưu thông tin detection mà còn lưu thêm snapshot camera metadata.

Các thông tin bao gồm:

- camera_id
- camera_code
- camera_name
- camera_location
- camera_source_type
- is_demo

Ý nghĩa của thay đổi này là đảm bảo dữ liệu vi phạm luôn giữ được ngữ cảnh gốc tại thời điểm phát hiện. Đây là yếu tố quan trọng khi hệ thống có nhiều camera và camera có thể bị đổi tên, đổi vị trí hoặc bị soft-delete sau này.

5.2. Bổ sung bbox dedup để giảm log trùng

Trong quá trình chạy tracking thực tế, ByteTrack đôi khi có thể đổi track_id của cùng một người nếu đối tượng bị che khuất, rời khỏi khung hình ngắn hoặc bị mất tracking tạm thời.

Điều này có thể khiến cùng một người bị ghi log nhiều lần.

Để xử lý, em đã bổ sung cơ chế bbox dedup dựa trên các thông số:

- VIOLATION_DEDUP_SECONDS
- VIOLATION_DEDUP_IOU_THRESHOLD
- VIOLATION_DEDUP_CENTER_DISTANCE

Cơ chế này giúp kiểm tra xem một vi phạm mới có quá gần về vị trí và thời gian so với vi phạm vừa ghi nhận trước đó hay không. Nếu có, hệ thống sẽ bỏ qua để tránh log trùng.

6. Cải thiện độ ổn định pipeline video

6.1. Tách release camera khỏi async lock

Trong quá trình switch camera, việc release camera có thể mất thời gian và nếu thực hiện bên trong async lock sẽ làm block event loop.

Để khắc phục, em đã tách release camera khỏi async lock và chuyển việc giải phóng tài nguyên sang background executor.

Điều này giúp:

- Backend không bị treo khi release camera
- Frontend nhận phản hồi nhanh hơn
- Switch camera ổn định hơn

6.2. Thêm pause event cho capture loop

Một vấn đề khác là trong lúc switch camera, capture loop có thể tiếp tục chạy và cố gắng mở nguồn mới khi handle cũ chưa release xong.

Để xử lý, em đã bổ sung pause event cho capture loop.

Cơ chế này giúp pipeline tạm dừng đúng thời điểm, chờ nguồn cũ giải phóng xong rồi mới tiếp tục với nguồn mới.

Nhờ đó, hệ thống hạn chế được lỗi tranh chấp tài nguyên camera, đặc biệt khi dùng webcam hoặc RTSP trên Windows.

6.3. Clear state khi switch/start/stop

Khi chuyển camera hoặc dừng stream, các state cũ như track history hoặc recent violation events nếu không được xóa sẽ ảnh hưởng tới nguồn mới.

Trong tuần này, em đã đảm bảo clear các state sau khi switch/start/stop:

- track_history
- logged_ids
- recent_violation_events
- spatial_violation_history

Điều này giúp mỗi phiên camera bắt đầu với trạng thái sạch, tránh việc violation hoặc tracking từ camera cũ ảnh hưởng sang camera mới.

7. Kết quả đạt được trong tuần 12

Sau khi hoàn thành các công việc trong tuần thứ mười hai, các kết quả chính đạt được bao gồm:

- Xây dựng thành công hệ thống Audit Log cho các hành động quan trọng
 - Bổ sung API truy xuất Audit Log với filter và pagination
 - Xây dựng trang Audit Logs trên Frontend cho Admin
- Xây dựng module Camera Management trong SQL Database
 - Bổ sung API tạo, sửa, bật/tắt, xóa mềm và test connection camera
 - Xây dựng giao diện Camera Management cho Admin
- Bổ sung Demo Mode dưới dạng video_file camera source
- Bổ sung bbox dedup để giảm log trùng
- Cải thiện độ ổn định khi switch/start/stop camera

Những kết quả này giúp hệ thống tiến gần hơn tới mô hình production, có khả năng quản trị camera, truy vết thao tác người dùng và phục vụ demo đồ án ổn định hơn.

8. Khó khăn gặp phải và cách khắc phục

Trong tuần này, quá trình bổ sung Audit Log, Camera Management và Demo Mode đã phát sinh nhiều thách thức liên quan đến thiết kế dữ liệu, bảo mật thông tin camera, xử lý tài nguyên video và đảm bảo dữ liệu thống kê không bị sai lệch. Các vấn

đề này đòi hỏi phải can thiệp đồng thời vào cả Backend, Frontend, Database và pipeline AI realtime.

7.1. Vấn đề bảo mật RTSP URL

Thách thức:

RTSP URL có thể chứa thông tin nhạy cảm như username, password hoặc địa chỉ camera nội bộ. Nếu trả source_url trực tiếp về Live Monitoring cho, hệ thống có nguy cơ lộ thông tin nhạy cảm đó.

Giải pháp:

Live selector chỉ trả metadata cần thiết như code, name, location, source_type, last_status và runtime_status. Frontend chỉ gửi camera code/id khi switch camera, còn Backend chịu trách nhiệm tra source_url trong database.

Kết quả:

Cách xử lý này giúp bảo mật nguồn camera tốt hơn mà vẫn đảm bảo Frontend có đủ thông tin để hiển thị.

7.2. Dữ liệu demo có thể làm sai lệch Analytics

Thách thức:

Demo Mode rất cần thiết cho quá trình báo cáo, nhưng nếu dữ liệu demo được tính chung vào Analytics thì báo cáo sẽ không còn phản ánh dữ liệu thực tế.

Giải pháp:

Em bổ sung trường is_demo cho violation sinh ra từ demo source. Sau đó, Analytics và Traffic Stats mặc định loại bỏ các bản ghi demo. Đồng thời, demo video cũng có cooldown riêng để tránh việc video loop liên tục tạo nhiều bản ghi trùng vào MongoDB.

Kết quả:

Demo Mode có thể phục vụ báo cáo mà không làm sai lệch dữ liệu vận hành thật.

7.3. Log trùng do ByteTrack đổi track_id

Thách thức:

Trong điều kiện thực tế, ByteTrack không phải lúc nào cũng giữ nguyên track_id cho cùng một người. Khi đối tượng bị che khuất hoặc rời khung hình ngắn, hệ thống có thể cấp track_id mới. Điều này khiến cùng một người có thể bị ghi nhận nhiều lần.

Giải pháp:

Em bổ sung bbox dedup dựa trên thời gian, IoU và khoảng cách tâm bounding box. Nếu một detection mới quá gần với detection vừa ghi trước đó, hệ thống sẽ bỏ qua để tránh log trùng.

Kết quả:

Cách này giúp giảm nhiều dữ liệu nhưng vẫn giữ lại Track Voting làm cơ chế xác nhận chính.

8. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã bổ sung Audit Log, Camera Management và Demo Mode trong tuần này, hệ thống đã hoàn thiện hơn về mặt quản trị và khả năng demo. Trong tuần tiếp theo, em sẽ tập trung vào việc kiểm thử toàn diện, hoàn thiện sản phẩm cuối cùng và chuẩn bị cho giai đoạn báo cáo cuối kỳ.

8.1. Kiểm thử toàn bộ hệ thống

Trong tuần tiếp theo, em sẽ kiểm tra lại toàn bộ các luồng chính của hệ thống, bao gồm:

- Đăng nhập, phân quyền và route protection
- Camera Management
- Audit Logs
- Live Monitoring
- Switch camera
- Demo Mode
- Violation Review Workflow
- Analytics
- Export Excel và AI Feedback Dataset

Mục tiêu là đảm bảo các chức năng mới không gây lỗi cho các module cũ và toàn bộ hệ thống hoạt động ổn định khi kết hợp với nhau.

8.2. Hoàn thiện tài liệu và báo cáo cuối kỳ

Bên cạnh việc kiểm thử kỹ thuật, em sẽ tập trung hoàn thiện các tài liệu phục vụ báo cáo, bao gồm:

- Báo cáo tổng kết dự án
- Video demo hệ thống
- Hướng dẫn cài đặt Backend và Frontend
- Hướng dẫn sử dụng các chức năng chính

Đây là giai đoạn cuối nhằm đảm bảo sản phẩm không chỉ hoạt động tốt mà còn được trình bày rõ ràng, đầy đủ và thuyết phục trong buổi bảo vệ.